Pau Casesnoves
Centre Integrat de Formació Professional

PROVA D'AVALUACIÓ UNITAT 1

Curs	26/27	**Grup**	IFC33C	**Data**		**Qualificació**	
Mòdul	Desenvolupament web en entorn servidor						
Alumne/a							

El servei de reprografia de la biblioteca — Solucionari

Versió amb respostes de referència i criteris de correcció.

Els enunciats i puntuacions corresponen al PDF revisat pel professor el 14/10/2026. Les respostes ometen imports i paquets com permet la prova. Els subapartats es valoren separadament; el barem sobre 10 conserva l'estat provisional de la proposta.

La biblioteca desenvolupa amb Java i Spring Boot un servei d'impressió. Cada petició HTTP pot incloure diversos documents. El sistema prepara cada document, en gestiona la impressió i registra la traça de l'operació.

Respon amb codi Java, anotacions i justificacions. Pots ometre imports, paquets i codi repetitiu, i simular operacions amb missatges. No cal implementar controladors. Els apartats es valoren separadament: suposa correctes les altres peces. Pots continuar al revers indicant l'apartat.

1. Preparació de documents — 1,5 punts

La interfície `PreparadorDocument`, amb el mètode `String preparar(String nomDocument)`, té dues implementacions gestionades per Spring: `PreparadorText` i `PreparadorImatge`.

Una dependència de tipus `PreparadorDocument` sense preferència concreta ha de rebre el de text. `ServeiCartelleria`, compartit per tota l'aplicació, necessita el d'imatge.

a) Escriu la interfície i les declaracions de les implementacions, amb anotacions i signatures; omet el cos de `preparar`. **(0,75 punts)**

b) Escriu la declaració, el camp i el constructor de `ServeiCartelleria`. Explica quin preparador rebria un servei sense preferència concreta. **(0,75 punts)**

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

Resposta de referència

a) Interfície i implementacions (0,75 punts). Els cossos següents només simulen la preparació; a la resposta d'examen es poden ometre, tal com indica l'enunciat.

```
public interface PreparadorDocument {
    String preparar(String nomDocument);
}
@Component
@Primary
public class PreparadorText implements PreparadorDocument {
    @Override
    public String preparar(String nomDocument) {
        return "Text preparat: " + nomDocument;
    }
}
@Component
public class PreparadorImatge implements PreparadorDocument {
    @Override
    public String preparar(String nomDocument) {
        return "Imatge preparada: " + nomDocument;
    }
}
```

b) Selecció al servei (0,75 punts).

```
@Service
public class ServeiCartelleria {
    private final PreparadorDocument preparador;

    @Autowired
    public ServeiCartelleria(
            @Qualifier("preparadorImatge") PreparadorDocument preparador) {
        this.preparador = preparador;
    }
}
```

Un servei que només demana `PreparadorDocument` rep `PreparadorText`, perquè és el candidat amb `@Primary`. El qualificador del constructor de `ServeiCartelleria` selecciona l'altre candidat. `preparadorImatge` és el nom que Spring assigna per defecte al component `PreparadorImatge`. `@Service` és singleton per defecte.

Correcció: a) 0,25 pel contracte i declaracions compatibles, 0,25 pel registre dels candidats i 0,25 per la preferència de text; b) 0,25 per servei i constructor, 0,25 per selecció i 0,25 per justificació. Admetre un nom explícit de bean amb qualificador coherent i l'omissió d'`@Autowired` en un constructor únic.

2. Marca d'aigua opcional — 1,5 punts

`ServeiPreparacio` necessita sempre un `PreparadorDocument`. Algunes instal·lacions també disposen d'un component `MarcaAigua`, amb el mètode `String aplicar(String contingut)`, però en altres instal·lacions aquest component no està registrat a Spring.

L'aplicació ha d'arrencar en tots dos casos. El mètode `String preparar(String nomDocument)` de `ServeiPreparacio` obté primer el contingut del preparador. Si hi ha marca d'aigua, l'aplica; si no, retorna el contingut sense modificar.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

a) Escriu la declaració del servei, els camps i el constructor per a la dependència imprescindible. **(0,5 punts)**

b) Mostra com rep la col·laboració de `MarcaAigua` sense exigir que el component existeixi. **(0,5 punts)**

c) Implementa `preparar` i explica què ocorre amb la injecció i amb el resultat quan no hi ha cap component `MarcaAigua`. **(0,5 punts)**

Resposta de referència

a), b) i c) Servei, injecció opcional i operació (0,5 punts cadascun). S'utilitza el contracte de `MarcaAigua` donat a l'enunciat; no cal implementar-lo.

```
@Service
public class ServeiPreparacio {
    private final PreparadorDocument preparador;
    private MarcaAigua marcaAigua;

    @Autowired
    public ServeiPreparacio(PreparadorDocument preparador) {
        this.preparador = preparador;
    }

    @Autowired(required = false)
    public void setMarcaAigua(MarcaAigua marcaAigua) {
        this.marcaAigua = marcaAigua;
    }

    public String preparar(String nomDocument) {
        String contingut = preparador.preparar(nomDocument);
        if (marcaAigua != null) {
            return marcaAigua.aplicar(contingut);
        }
        return contingut;
    }
}
```

El preparador és imprescindible i es rep per constructor. Amb els components de l'apartat 1, serà el de text. Si no existeix el bean `MarcaAigua`, Spring omet la crida al setter opcional: el camp continua a `null`, el context pot arrencar i `preparar` retorna el contingut sense marca. Si existeix, el setter rep el bean i s'aplica la marca al contingut preparat.

Correcció: 0,5 pel servei i la dependència obligatòria, 0,5 per configurar la injecció opcional i 0,5 pel comportament i la justificació de l'absència. No basta un setter sense configurar l'opcionalitat ni basta fer opcional la injecció si el mètode després usa la referència sense comprovar-la. Es poden admetre alternatives vàlides d'injecció opcional si respecten els requisits i se'n justifica el comportament.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

3. Traça d'una operació — 1,5 punts

Cada petició HTTP que gestiona impressions necessita un `ContextImpressio` amb un identificador aleatori, accessible amb `String getId()`. Tots els documents i missatges de la mateixa petició han de compartir aquest identificador. Una altra petició ha de tenir un identificador diferent, encara que provingui del mateix navegador.

`ServeiImpressio` i `RegistreImpressio` són serveis compartits per tota l'aplicació i es creen a l'arrencada, quan encara no hi ha cap petició HTTP. Tots dos reben directament una dependència de tipus `ContextImpressio` al constructor i la conserven en un camp. Durant l'atenció de la petició consulten `getId()` sobre aquesta dependència; no es passen l'identificador com a paràmetre.

a) Escriu `ContextImpressio`, amb la creació de l'identificador, el getter i la configuració necessària perquè satisfaci aquests requisits. Pots emprar `UUID.randomUUID().toString()`. **(0,5 punts)**

b) Escriu els camps i constructors pertinents dels dos serveis, amb les seves anotacions. No cal escriure la resta dels mètodes. **(0,5 punts)**

c) Justifica per què els serveis es poden crear abans de la primera petició i, després, consultar el context corresponent. Indica quins identificadors veuen dins una mateixa petició i en dues peticions diferents. **(0,5 punts)**

Resposta de referència

a) Context de petició (0,5 punts).

```
@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class ContextImpressio {
    private final String id = UUID.randomUUID().toString();

    public String getId() {
        return id;
    }
}
```

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

b) Injecció als dos serveis (0,5 punts). Aquests fragments mostren els camps i constructors demanats; les consultes a `getId()` es fan durant la petició, no als constructors.

```
@Service
public class ServeiImpressio {
    private final ContextImpressio context;

    @Autowired
    public ServeiImpressio(ContextImpressio context) {
        this.context = context;
    }
}
@Service
public class RegistreImpressio {
    private final ContextImpressio context;

    @Autowired
    public RegistreImpressio(ContextImpressio context) {
        this.context = context;
    }
}
```

c) Justificació (0,5 punts). Spring injecta un proxy que els serveis poden conservar des de l'arrencada, encara que no hi hagi una petició activa. Quan un servei crida `context.getId()` durant una petició HTTP, el proxy localitza el `ContextImpressio` d'aquella petició i hi delega. Els dos serveis consulten el mateix objecte de context dins una petició i obtenen el mateix identificador. En una altra petició es crea un altre context amb un altre UUID, també si és el mateix navegador. Els serveis continuen sent singletons: el camp conserva el proxy, no es reinjecta a cada petició.

Correcció: a) 0,25 per identitat i abast de petició i 0,25 pel proxy; b) 0,25 per cada servei; c) 0,25 per explicar la delegació i l'arrencada i 0,25 per distingir les identitats. També és vàlid `@Component` amb `@RequestScope`, que configura el proxy de classe. Un `@Scope("request")` sense proxy no resol la injecció directa en aquests singletons. La classe i `getId()` no han de ser `final`; el camp receptor sí que pot ser-ho. Consultar `getId()` al constructor intentaria accedir al context quan encara no hi ha petició activa.

4. Un treball per document — 1,5 punts

Per a cada document, `ServeiImpressio` ha d'obtenir un `TreballImpressio` nou i gestionat per Spring. Dos documents de la mateixa petició necessiten dos treballs diferents, encara que comparteixin el context de l'apartat anterior. El servei continua sent una única instància compartida.

La classe ja té aquest codi; copia només la declaració i les anotacions que hi afegiries:

```
public class TreballImpressio {
    private final String id = UUID.randomUUID().toString();

    public String executar(String nomDocument) {
        return id + ": " + nomDocument;
    }
}
```

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

La petició dels treballs al contenidor s'ha de concentrar en una classe `FabricaTreballs`, amb el mètode `TreballImpressio nouTreball()`. El servei rep aquesta fàbrica i la utilitza cada vegada que processa un document. Ni el servei ni la fàbrica han de construir manualment els treballs amb `new`.

a) Configura `TreballImpressio` perquè cada petició d'aquest objecte al contenidor obtingui una instància nova. **(0,5 punts)**

b) Escriu `FabricaTreballs` i el fragment de `ServeiImpressio` que rep la fàbrica i implementa `String imprimirDocument(String nomDocument)`, obtenint un treball i retornant-ne el resultat d'`executar`. **(0,75 punts)**

c) Explica per què injectar directament un únic `TreballImpressio` al constructor del servei i conservar-lo no compliria el requisit. **(0,25 punts)**

Resposta de referència

a) Configuració del treball (0,5 punts). El cos és el que proporciona l'enunciat; la resposta demanada només n'ha de copiar la declaració i afegir les anotacions.

```
@Component
@Scope("prototype")
public class TreballImpressio {
    private final String id = UUID.randomUUID().toString();

    public String executar(String nomDocument) {
        return id + ": " + nomDocument;
    }
}
```

b) Factory i ús al servei (0,75 punts).

```
@Component
public class FabricaTreballs {
    private final ObjectProvider<TreballImpressio> provider;

    @Autowired
    public FabricaTreballs(ObjectProvider<TreballImpressio> provider) {
        this.provider = provider;
    }

    public TreballImpressio nouTreball() {
        return provider.getObject();
    }
}
```

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

La versió següent amplia `ServeiImpressio` de l'apartat 3 amb la fàbrica i el mètode demanat. És el mateix servei, amb un únic constructor que rep totes dues dependències; no s'han de registrar dues classes del mateix nom.

```
@Service
public class ServeiImpressio {
    private final ContextImpressio context;
    private final FabricaTreballs fabrica;

    @Autowired
    public ServeiImpressio(ContextImpressio context,
                           FabricaTreballs fabrica) {
        this.context = context;
        this.fabrica = fabrica;
    }

    public String imprimirDocument(String nomDocument) {
        TreballImpressio treball = fabrica.nouTreball();
        return treball.executar(nomDocument);
    }
}
```

c) Justificació (0,25 punts). Injectar un prototype directament al constructor d'un singleton resol aquella dependència una vegada, quan es construeix el servei. El servei conservaria aquell treball i el reutilitzaria. Amb la fàbrica es demana el bean dins cada operació: `getObject()` obté una instància nova perquè `TreballImpressio` és prototype. Dos documents de la mateixa petició comparteixen el context, però disposen de dos treballs diferents. La fàbrica també pot ser singleton perquè conserva el proveïdor i no conserva un treball per reutilitzar.

Correcció: 0,5 per registrar el treball amb l'abast adequat; a b), 0,25 per declarar i injectar el provider, 0,25 per fer `getObject()` a `nouTreball()` i 0,25 per injectar i usar la fàbrica dins el mètode del servei; 0,25 per la justificació. Acceptar `return fabrica.nouTreball().executar(nomDocument);`. Valorar per separat l'scope i la petició al contenidor: el provider respecta l'scope i no converteix un singleton en un prototype.

5. Connexió amb la impressora — 1,5 punts

`MotorImpressora` és un component compartit que estableix una connexió costosa durant la seva construcció. Ofereix el mètode `String enviar(String contingut)`. La connexió només s'ha de preparar quan s'envia la primera impressió; les impressions següents reutilitzen el mateix component.

`ServeiEnviament` també és compartit i s'ha de crear a l'arrencada. Rep `MotorImpressora` pel constructor. La creació del servei no ha de provocar la connexió amb la impressora; el seu mètode `String enviar(String contingut)` delega en el motor.

a) Escriu la declaració de `MotorImpressora` amb la configuració necessària. Simula la connexió amb un missatge al constructor i implementa `enviar` retornant un text. **(0,5 punts)**

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

b) Escriu `ServeiEnviament`, incloent el camp, el constructor i el mètode de delegació. **(0,5 punts)**

c) Explica quan es construeix el motor i per què la configuració de la injecció al servei és rellevant per complir el requisit. **(0,5 punts)**

Resposta de referència

a) Component d'inicialització costosa (0,5 punts).

```
@Component
@Lazy
public class MotorImpressora {
    public MotorImpressora() {
        System.out.println("Preparant la connexió amb la impressora");
    }

    public String enviar(String contingut) {
        return "Enviat a la impressora: " + contingut;
    }
}
```

b) Servei disponible a l'arrencada (0,5 punts).

```
@Service
public class ServeiEnviament {
    private final MotorImpressora motor;

    @Autowired
    public ServeiEnviament(@Lazy MotorImpressora motor) {
        this.motor = motor;
    }

    public String enviar(String contingut) {
        return motor.enviar(contingut);
    }
}
```

c) Justificació (0,5 punts). `@Lazy` al component evita la seva preinstanciació habitual com a singleton a l'arrencada. `@Lazy` al paràmetre permet injectar al servei una referència diferida, sense construir el motor en aquell moment. La primera crida a `motor.enviar(...)` resol la dependència, construeix el motor i prepara la connexió simulada. Les crides següents reutilitzen el mateix motor singleton.

Correcció: a) 0,25 per component i simulació i 0,25 per ajornament del bean; b) 0,25 per estructura i delegació i 0,25 per ajornament al punt d'injecció; c) 0,5 per explicar primer ús, reutilització i funció de la injecció diferida. Posar `@Lazy` només al component permetria que una injecció directa el fes necessari a l'arrencada; posar-la només al punt d'injecció no evita la preinstanciació habitual d'un component no diferit. No es demana cap espera ni connexió real.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

6. Preparació i alliberament de recursos — 1,5 punts

El component compartit `RegistreRecursos` gestiona un recurs propi on anota incidències de les impressions. Una vegada construït i amb les dependències ja proporcionades, ha de preparar aquest recurs abans de començar a prestar servei. Quan el context de Spring es tanqui ordenadament, l'ha de tancar.

Pots simular la preparació i el tancament amb missatges; no cal programar l'accés a fitxers.

a) Escriu la declaració del component i el mètode que prepara el recurs en el moment indicat. **(0,5 punts)**

b) Escriu el mètode que allibera el recurs en el tancament ordenat. **(0,5 punts)**

c) Explica en quin moment s'executa cadascun dels dos mètodes. S'executarien abans i després de cada impressió? Justifica la resposta. **(0,5 punts)**

Resposta de referència

a) i b) Preparació i tancament (0,5 punts cadascun).

```
@Component
public class RegistreRecursos {
    @PostConstruct
    public void prepararRecurs() {
        System.out.println("Recurs de registre preparat");
    }

    @PreDestroy
    public void tancarRecurs() {
        System.out.println("Recurs de registre tancat");
    }
}
```

Les anotacions són `jakarta.annotation.PostConstruct` i `jakarta.annotation.PreDestroy`. El component és singleton per defecte; no cal declarar-ho explícitament.

c) Justificació (0,5 punts). `prepararRecurs()` s'executa durant la inicialització de la instància, després de proporcionar les dependències i abans del seu ús habitual. `tancarRecurs()` s'executa quan Spring destrueix el singleton durant el tancament ordenat del context. Aquests mètodes no s'executen abans i després de cada impressió: estan vinculats al cicle de vida del component, no a cada crida de negoci.

Correcció: 0,5 per cada callback configurat al moment correcte i 0,5 per la justificació. Admetre altres noms de mètode amb signatures i anotacions correctes. Obrir el recurs al constructor no evidencia la fase d'inicialització amb dependències ja proporcionades que demana el cas; tancar-lo al final d'una impressió tampoc satisfà el requisit.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

7. Data i hora dels registres — 1 punt

El component `SegellTemps` ha de rebre pel constructor un `java.time.format.DateTimeFormatter` amb el patró `dd/MM/yyyy HH:mm:ss`. La classe del formatador pertany a la biblioteca Java i no la pots modificar.

La manera d'obtenir el formatador ha de quedar centralitzada en una classe `ConfiguracioReprografia`. Spring ha de gestionar l'objecte resultant i proporcionar-lo als components que el necessiten; `SegellTemps` no ha de crear el seu formatador.

Per crear l'objecte, l'API Java ofereix `DateTimeFormatter.ofPattern(String patro)`. Per obtenir la data i l'hora actuals amb aquest format, pots usar `LocalDateTime.now().format(formatador)`.

a) Escriu `ConfiguracioReprografia` amb el codi necessari perquè el formatador configurat quedi disponible al contenidor. **(0,5 punts)**

b) Escriu `SegellTemps`, amb el camp, la injecció i el mètode `String ara()`. Explica on canviaries el patró si la biblioteca volgués una altra presentació de la data. **(0,5 punts)**

Resposta de referència

a) Registre del formatador extern (0,5 punts).

```
@Configuration
public class ConfiguracioReprografia {
    @Bean
    public DateTimeFormatter formatador() {
        return DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
    }
}
```

b) Component receptor (0,5 punts).

```
@Component
public class SegellTemps {
    private final DateTimeFormatter formatador;

    @Autowired
    public SegellTemps(DateTimeFormatter formatador) {
        this.formatador = formatador;
    }

    public String ara() {
        return LocalDateTime.now().format(formatador);
    }
}
```

`@Configuration` identifica la classe de configuració i `@Bean` registra l'objecte retornat pel mètode. Spring proporciona aquest formatador a `SegellTemps` pel constructor. El patró es canvia al mètode `formatador()` de `ConfiguracioReprografia`; el component receptor no necessita canviar.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.

Correcció: a) 0,25 per configuració i registre del resultat i 0,25 pel tipus i patró correctes; b) 0,25 per component i injecció i 0,25 per `ara()` i localització del canvi. S'admet un altre nom de mètode de configuració coherent. No és necessari un qualificador si només hi ha un bean `DateTimeFormatter`. Crear el formatador dins `SegellTemps` no compleix el requisit de gestió i configuració centralitzada.

Criteris de qualificació: Solucionari: punts indicats a cada apartat. Total: 10.